

Large-Eddy Simulation of Turbulent Flow Past a Sphere

Physics, Boundary Conditions, Numerical Methods,
Algorithm and Pseudocode

Repository: `LES_smagorinsky / working/`

Solver module: `les_sphere_flow.py`

Method: Cell-centred Finite Volume + Smagorinsky LES
+ Artificial Compressibility pressure coupling

This document explains the simulation from first principles: the continuum equations, why LES is used, the mesh and boundary-condition design, the discrete finite-volume operators, the time-stepping algorithm, and the complete pseudocode as implemented.

Contents

1. Problem statement and physical setup
 2. Continuum equations of incompressible flow
 3. Why turbulence modelling / LES is needed
 4. Smagorinsky LES closure
 5. Geometry, mesh and nondimensional parameters
 6. Boundary conditions (and design rationale)
 7. Numerical methods (finite-volume discretisation)
 8. Time-stepping and pressure–velocity coupling
 9. Full algorithm flowchart (with substeps)
 10. Detailed pseudocode
 11. Implementation map (code $\leftrightarrow$ theory)
 12. Outputs and post-processing
 13. Limitations and recommended next improvements
- Appendix A. Symbol table
- Appendix B. Typical run commands

1. Problem statement and physical setup

We simulate three-dimensional incompressible flow of a Newtonian fluid past a fixed sphere. The sphere is immersed in a rectangular channel-like domain. Fluid enters from the left (inlet), interacts with the sphere, forms a wake, and leaves through the right (outlet). The goal is to capture the large-scale unsteady / turbulent features of the wake using Large-Eddy Simulation (LES), while modelling the unresolved small scales with an eddy viscosity.

Physical picture

- Upstream: nearly uniform freestream velocity U_∞ .
- Near sphere: stagnation at the front, acceleration around the shoulders, shear layers, separation.
- Downstream: wake with velocity deficit and unsteady vortical structures.
- Outer walls: treated as free-slip so they do not impose strong channel boundary layers that would artificially disturb the sphere flow.

The immersed body in this repository is a **sphere** (radius $R = 0.5$), even if some notes historically said “cylinder”. A mid-plane slice looks circular, which can be confused with 2D cylinder flow.

2. Continuum equations of incompressible flow

For constant-density incompressible flow, mass conservation reduces to a divergence-free constraint on velocity, and momentum is the Navier–Stokes equation.

Continuity (mass):

$$\nabla \cdot \mathbf{U} = 0$$

Momentum:

$$\partial \mathbf{U} / \partial t + \nabla \cdot (\mathbf{U} \otimes \mathbf{U}) = -(1/\rho) \nabla p + \nabla \cdot [\nu (\nabla \mathbf{U} + (\nabla \mathbf{U})^T)] + \mathbf{f}$$

Here $\mathbf{U}$ is velocity, p pressure, ρ density, ν kinematic viscosity, and $\mathbf{f}$ any body force (taken as zero). The convective term transports momentum; the viscous term dissipates it; pressure enforces incompressibility.

Important practical point: pressure is not an independent thermodynamic variable here; it is a Lagrange multiplier (or constraint field) that keeps $\nabla \cdot \mathbf{U} \approx 0$ at every instant.

3. Why turbulence modelling / LES is needed

At Reynolds number $Re = U_\infty D / \nu \approx 1000$ (with $D = 2R = 1$), the wake becomes unsteady and can support a broad range of eddy sizes. Direct Numerical Simulation (DNS) would require resolving all scales down to the Kolmogorov length, which is expensive on a coarse unstructured mesh.

Strategy comparison

Approach	Resolved	Modelled
DNS	All scales	None
LES	Large energy-containing eddies	Subgrid scales via ν_t
RANS	Mean flow only	All turbulence

LES is a compromise: the mesh resolves the large wake structures; the Smagorinsky model dissipates energy that would otherwise cascade into unresolved scales.

4. Smagorinsky LES closure

After spatial filtering (implicitly done by the mesh), the momentum equation contains an unresolved subgrid stress. The Smagorinsky model approximates its effect by an eddy viscosity ν_t .

Strain-rate tensor:

$$S = \frac{1}{2} (\nabla U + (\nabla U)^T), \quad |S| = \sqrt{2 S_{ij} S_{ij}}$$

Filter width (cell-based):

$$\Delta = V^{1/3} \quad (V = \text{cell volume})$$

Eddy viscosity:

$$\nu_t = (C_s \Delta)^2 |S|$$

Effective viscosity used in diffusion:

$$\nu_{\text{eff}} = \nu + \nu_t$$

In the code, $C_s = 0.1$ by default, Δ is capped, and ν_t is clipped for stability. Physically, ν_t is large in high-shear regions (near the sphere / shear layers) and small in quiet freestream.

LES role in this simulation

1. Compute cell velocity gradients from face fluxes (Green–Gauss).
2. Form $|S|$ and Δ .
3. Update ν_t every time step.
4. Insert ν_{eff} into the viscous flux discretisation.

Thus LES does not replace Navier–Stokes; it augments molecular viscosity with a scale-aware turbulent viscosity.

5. Geometry, mesh and nondimensional parameters

Geometry (mesh generator defaults)

Quantity	Value
Domain box $L \times W \times H$	$10 \times 5 \times 5$
Sphere centre	(3.0, 2.5, 2.5)
Sphere radius R / diameter D	0.5 / 1.0
Inlet plane	$x \approx 0$
Outlet plane	$x \approx L$

Flow parameters

Parameter	Symbol	Default
Freestream velocity	U_∞	1.0
Density	ρ	1.0
Kinematic viscosity	ν	1e-3
Reynolds number	$Re = U_\infty D / \nu$	≈ 1000
Smagorinsky constant	C_s	0.1

Time step	Δt	1e-4 (typical)
Inlet TI	TI	~1–1.5%

The mesh is an unstructured tetrahedral finite-volume mesh stored in `processed_mesh.npz` (cell centres, volumes, faces, owner/neighbour addressing, boundary tags).

6. Boundary conditions (and design rationale)

Boundary conditions are applied on faces after interpolating cell values to faces. The design goal is that inlet/outlet treatment should not corrupt the sphere wake.

6.1 Inlet ($x = 0$)

Dirichlet velocity: $U = (U_\infty, 0, 0)$ plus mild random fluctuations proportional to $Tl \cdot U_\infty$. This seeds LES without imposing strong synthetic turbulence that would overwhelm the natural wake.

6.2 Outlet ($x = L$)

Convective (Orlanski-type) outflow for velocity:

$$\partial U / \partial t + U_c \partial U / \partial n \approx 0, \text{ with } U_c \approx U_\infty$$

Reverse flow is suppressed. Pressure is treated softly near the outlet (not a hard reflecting wall), so structures can leave the domain with reduced reflection.

6.3 Outer walls

Free-slip: normal velocity zero, tangential velocity retained ($U \leftarrow U - (U \cdot n)n$). Compared with no-slip walls, this reduces blockage and unwanted wall boundary layers.

6.4 Sphere surface (object)

No-slip: $U = 0$ on sphere faces. This is the physically correct viscous wall condition and is responsible for boundary-layer formation, separation, and wake generation.

Patch	Velocity BC	Pressure treatment
Inlet	Dirichlet U_∞ + mild TI	Soft / Neumann-like
Outlet	Convective Orlanski	Soft (non-reflecting)
Outer walls	Free-slip	Neumann-like
Sphere	No-slip	Neumann-like

7. Numerical methods (finite-volume discretisation)

The solver is a collocated, cell-centred finite-volume method on tetrahedra.

7.1 Mesh data structures

- Cells store U , p , vt at centroids.
- Faces store area vectors S_f (oriented out of owner), owner index, neighbour index (-1 if boundary).
- Boundary faces carry tags: inlet=1, outlet=2, walls=3, object=4.

7.2 Face interpolation

Internal faces use inverse-distance weights between owner and neighbour. Boundary faces start from the owner value and are then overwritten by the BC rules above.

7.3 Gradient (Green–Gauss)

$$(\nabla \phi)_P \approx (1/V_P) \sum_f \phi_f S_f$$

7.4 Convection

Volumetric flux $\phi_f = U_f \cdot S_f$. Convective transport uses first-order upwind on internal faces (donor cell depending on flux sign). This is dissipative but robust on coarse meshes.

7.5 Diffusion

Two-point viscous flux with coefficient $a_f \approx |S_f| / |d \cdot n|$, using ν_{eff} . Boundary diffusion uses the Dirichlet/free-slip face value.

8. Time-stepping and pressure–velocity coupling

A fully coupled incompressible projection (exact PISO/SIMPLE) is delicate on unstructured collocated meshes. This implementation uses an explicit momentum update followed by artificial-compressibility (AC) iterations to reduce divergence.

8.1 Momentum predictor

$$\mathbf{U}^* = \mathbf{U}^\blacksquare + \Delta t \left[(-\text{conv} + \text{diff})/\mathbf{V} - (1/\rho) \nabla p^\blacksquare \right]$$

8.2 Artificial compressibility sub-iterations

Inside each time step, several AC iterations are performed:

$$p \leftarrow p - \Delta t \, c^2 \, (\nabla \cdot \mathbf{U}),$$

then a flux-consistent velocity correction based on face pressure jumps is applied. This drives the discrete divergence down without requiring a fully converged Poisson solve every step.

8.3 Stabilisation

- Under-relaxation of velocity.
- Mild damping of sphere-adjacent cells.
- Velocity magnitude cap for robustness on coarse meshes / large Δt .

Interpretation: the method is a practical teaching/demo LES finite-volume solver. It prioritises robustness and clear BC/LES structure over textbook industrial CFD accuracy.

9. Full algorithm flowchart (with substeps)

```
[A] Load mesh (processed_mesh.npz)
    |
    v
[B] Initialize  $\mathbf{U}=(\mathbf{U}_\infty,0,0)$ ,  $p=0$ ,  $\nu_t=0$ 
    |
    v
[C] FOR step = 1 ... Nsteps
    |
    +--> [C1] Interpolate  $\mathbf{U}$  to faces
    |
    +--> [C2] Apply BCs on faces
    |       C2a Inlet Dirichlet + TI
    |       C2b Outlet Orlanski
    |       C2c Walls free-slip
    |       C2d Sphere no-slip
    |
    +--> [C3] LES Smagorinsky
    |       C3a Green-Gauss  $\nabla \mathbf{U}$ 
    |       C3b  $|\mathbf{S}|$ ,  $\Delta$ 
    |       C3c  $\nu_t=(Cs\Delta)^2|\mathbf{S}|$ 
    |       C3d  $\nu_{eff}=\nu+\nu_t$ 
    |
    +--> [C4] Momentum fluxes
    |       C4a Convective upwind
    |       C4b Viscous diffusion
    |       C4c Soft outlet pressure gradient
    |       C4d Explicit  $\mathbf{U}$  update
    |
    +--> [C5] AC pressure-velocity iterations ( $\times 4$ )
    |       C5a Re-apply BCs
    |       C5b Compute  $\nabla \cdot \mathbf{U}$ 
    |       C5c Update  $p$ 
    |       C5d Correct  $\mathbf{U}$  from  $\Delta p$  fluxes
```



```
|
+--> [C6] Under-relax / damp / clip
|
+--> [C7] Optional: save GIF frame
|
v
[D] Post-process: midplane, streamlines, wake, vt, GIF
```

Figure-style textual flowchart of the solver driver and one time step.

10. Detailed pseudocode

10.1 Top-level driver

```
ALGORITHM RunLES_FlowPastSphere
  INPUT: mesh file, SimConfig cfg
  OUTPUT: fields, plots, gif

  mesh ← LoadProcessedMesh(mesh file)
  U[:] ← (cfg.U_inf, 0, 0)
  p[:] ← 0
  nu_t[:] ← 0
  frames ← empty list

  N ← ceil(cfg.t_end / cfg.dt)
  FOR step = 1 TO N DO
    StepOnce(mesh, U, p, nu_t, cfg)
    t ← step * cfg.dt
    IF capture frame this step THEN
      frames.append(RenderMidplaneSpeed(U, t, step))
    END IF
    IF print this step THEN
      ReportDiagnostics(U, p, nu_t, t)
    END IF
  END FOR

  SaveFields(U, p, nu_t)
  PlotMidplane(U, p)
  PlotStreamlines(U)
  PlotWake(U)
  PlotNut(nu_t)
  IF frames not empty THEN SaveGIF(frames)
END ALGORITHM
```

10.2 One time step

```
PROCEDURE StepOnce(mesh, U, p, nu_t, cfg)
  # --- faces and BCs ---
  Uf ← InterpolateToFaces(U)
  Uf ← ApplyVelocityBCs(Uf, U, cfg)
  # inlet: Uinf + mild noise
  # outlet: Orlanski convective update
  # walls: remove normal component
  # sphere: set zero

  # --- LES ---
  gradU ← GreenGaussGradient(Uf, mesh)
  S ← 0.5*(gradU + transpose(gradU))
  S_mag ← sqrt(2 * Frobenius(S)^2)
  delta ← min(V**(1/3), 0.5)
  nu_t ← clip( (cfg.Cs*delta)**2 * S_mag , 0, 50*cfg.nu )
  nu_eff ← cfg.nu + nu_t

  # --- fluxes ---
  phi ← Uf · Sf
  conv ← UpwindDivergence(U, phi)          # Σ flux * U_upwind
  diff ← ViscousDiffusion(U, Uf, nu_eff)   # two-point face differences
  gradp ← GreenGaussScalar(p) with soft outlet face pressure
  Ustar ← U + cfg.dt * ( (-conv + diff)/V - gradp/cfg.rho )

  # --- artificial compressibility iterations ---
  U ← Ustar
  FOR k = 1 TO 4 DO
    Uf ← InterpolateToFaces(U); Uf ← ApplyVelocityBCs(...)
    phi ← Uf · Sf
    div ← DiscreteDivergence(phi)          # raw Σ fluxes
```

```

    p ← p - cfg.dt * c^2 * (div / V)
    p ← p - mean(p)
    CorrectVelocityFromPressureJump(U, p, cfg.dt, cfg.rho)
END FOR

DampSphereAdjacentCells(U)
U ← cfg.urf_u * U + (1-cfg.urf_u) * U_old
ClipSpeed(U, max=2.5*cfg.U_inf)
END PROCEDURE

```

10.3 Boundary-condition subroutine

```
PROCEDURE ApplyVelocityBCs(Uf, U_cell, cfg)
  # default boundary: copy owner
  FOR each boundary face f:
    Uf[f] ← U_cell[owner[f]]

  # inlet
  FOR each inlet face f:
    noise ← cfg.inlet_ti * cfg.U_inf * RandomNormal(3)
    noise_x ← 0.5 * noise_x
    Uf[f] ← (cfg.U_inf, 0, 0) + noise

  # outlet (Orlanski)
  Uc ← cfg.U_inf
  FOR each outlet face f:
    P ← owner[f]; n ← face_normal[f]
    dx ← | (xf - xP) · n | + eps
    Uf[f] ← Uf_out[f] - cfg.dt * Uc * (U_cell[P] - Uf_out[f]) / dx
    remove inflow component if (Uf[f]·n) < 0
    blend with interior extrapolation for robustness
    store Uf_out[f] ← Uf[f]

  # outer walls free-slip
  FOR each wall face f:
    n ← face_normal[f]
    u ← U_cell[owner[f]]
    Uf[f] ← u - (u·n) n

  # sphere no-slip
  FOR each object face f:
    Uf[f] ← 0
  RETURN Uf
END PROCEDURE
```

10.4 Smagorinsky subroutine

```
PROCEDURE UpdateSmagorinsky(Uf, mesh, cfg) -> nu_t
  gradU ← 0
  FOR each face f:
    contrib ← outer(Uf[f], Sf[f])
    accumulate +contrib to owner cell
    if internal: accumulate -contrib to neighbour cell
  FOR each cell P:
    gradU[P] ← gradU[P] / V[P]
    S ← symmetric part of gradU[P]
    S_mag ← sqrt(2)*||S||_F
    delta ← min(V[P]**(1/3), 0.5)
    nu_t[P] ← (cfg.Cs * delta)**2 * S_mag
  clip nu_t to [0, 50*cfg.nu]
  RETURN nu_t
END PROCEDURE
```

11. Implementation map (code ↔ theory)

Primary implementation file: **les_sphere_flow.py** (also copied under working/).

Theory component	Code entity
Parameters	SimConfig
Mesh + fields	SphereLESSolver.__init__
BCs	_apply_velocity_bcs
LES	_smagorinsky

One time step	step
Time loop	run
Streamlines / plots / GIF	plot_* , capture_frame, save_gif
Working entry point	working/run.py

12. Outputs and post-processing

After the time loop, the solver writes both quantitative fields and visual products.

- **les_sphere_fields.npz** — cell centres, U , p , vt .
- **flow_past_sphere_midplane.png** — mid-plane speed/pressure (with streamline overlay).
- **flow_past_sphere_streamlines.png** — full-domain and zoomed streamlines.
- **flow_past_sphere_wake.png** — centreline wake recovery.
- **flow_past_sphere_nut.png** — eddy-viscosity map (LES activity).
- **flow_past_sphere.gif** — animated mid-plane speed.

These are the practical “answers” of the simulation: they show whether the freestream approaches the sphere correctly, whether a wake forms, and whether LES viscosity activates in shear regions.

13. Limitations and recommended next improvements

This solver is intentionally transparent and educational. Current limitations include:

- First-order upwind convection is numerically dissipative.
- Artificial compressibility is an approximate substitute for a full PISO Poisson projection.
- Mesh resolution may be insufficient for high-fidelity LES at $Re = 1000$.
- No wall-damping / dynamic Smagorinsky / WALE model yet.
- No force coefficient history (C_d , C_l) or 3D Q-criterion visualisation yet.

Natural extensions

1. Replace AC by a true pressure-Poisson / PISO corrector with Rhie–Chow interpolation.
2. Use higher-order convection (e.g. limited linear) and adaptive time stepping.
3. Refine the near-sphere and wake mesh aggressively.
4. Add C_d/C_l monitors and probe spectra in the wake.
5. Export VTK for ParaView volume rendering.

Appendix A. Symbol table

Symbol	Meaning
U , U_f	Cell and face velocities
p	Pressure
ρ , ν	Density, molecular kinematic viscosity
vt , ν_{eff}	Eddy viscosity, effective viscosity
C_s , Δ , $ S $	Smagorinsky constant, filter width, strain magnitude
S_f , V	Face area vector, cell volume
$\phi = U_f \cdot S_f$	Volumetric face flux
U_∞ , Re	Freestream speed, Reynolds number
Δt	Time step

Appendix B. Typical run commands

```
# from working/  
py -m pip install -r requirements.txt
```

```
py run.py --quick          # short demo + GIF + plots
py run.py                  # longer output generation

# notebook
py -m jupyter lab
# open run_les_sphere.ipynb and Run All
```

Output directory: working/outputs/. Theory PDF location: working/docs/.

*End of document. This PDF is intended as a companion to the runnable working/ package and the source file
les_sphere_flow.py.*